

AN1.9 - Memory Contract + MemoryGateway (A7)

Sprint: AT24 AI Agents - Agent Framework Foundation **Step:** A7 - Memory Contract + MemoryGateway **Depends on:** AF-v1 memory contract (A1), Persistence (A3), Runtime (A4), Integrity (A6 / G06) **Gate:** G07 - the memory **authority and governance boundary**. Not a Knowledge/RAG system.

Two locks honoured: RUN_STATE is execution state, not memory. And an agent can **never silently promote its own output into trusted long-term knowledge** - an agent-derived write to a knowledge layer is forced to pending_approval, never active.

1. What was built

1.1 Contract - additive to types/agent-framework/memory-contract.ts (AF-v1 unchanged)

The 7 layers were already locked in A1. A7 adds the governance machinery:

- TABLE_MEMORY_LAYERS (the 6 table-backed layers; RUN_STATE excluded) and MEMORY_KNOWLEDGE_LAYERS (LONG_TERM, RESEARCH_MEMORY, STRATEGY_MEMORY).
- MEMORY_MAX_VALUE_BYTES = 16 KiB - the size bound.
- MemoryOrigin = user | tool | agent-derived | system; MemoryProvenance (origin, producer, runId?, stepId?, createdAt); MemoryRecordStatus = active | pending_approval | expired.
- MemoryWriteRequest (+ agentType denormalised so agent-type reads work) and MemoryReadQuery.
- resolveWriteDecision(policy, layer, origin) - the core rule, deterministic:
- a layer not in policy.layers -> "deny"
- base = policy.writePolicy[layer] ?? "deny" (default-deny)
- knowledge layer + agent-derived origin -> forced "approval" even if the policy says allow
- validateMemoryWriteRequest() - required fields, value <= 16 KiB + JSON-serializable, retention shape, "run" retention needs a runId, provenance shape, RUN_STATE rejected.

Conceptual mapping (owner's G06 model -> AF-v1): WORKING~==SHORT_TERM, EPISODIC~==PERFORMANCE_MEMORY, SEMANTIC~==LONG_TERM/RESEARCH_MEMORY, PROCEDURAL~==(no AF-v1 layer yet - additive when a real need appears), USER/AGENT/SHARED = the readPolicy scopes (own/agent-type/user-global), not layers. If the owner wants the vocabulary renamed, that is a one-line additive enum change + an ADR.

1.2 services/agent-framework/memory/

File	Role
memory-store.ts	The MemoryStore port the gateway depends on (insert / query / getById / markStatus / clearForUser) + StoredMemoryRecord. The gateway never touches Prisma directly.
in-memory-store.ts	InMemoryStore - process-local, deterministic (newest-first). Tests + a stand-in until the migration is applied.
prisma-memory-store.ts	PrismaMemoryStore - the real AgentMemoryRecord table. agentType is carried in provenance.agentType. Functional once the A7 migration is applied.
memory-gateway.ts	MemoryGateway - read() / write() / approvePending(). The one authority.
index.ts	server-only barrel.

1.3 MemoryGateway

```
read(query, policy)
|-- layer == RUN_STATE      -> deny "execution state, not memory"
|-- layer not in policy.layers -> deny "not granted"
|-- readScope = policy.readPolicy[layer] ?? "own"
|   own      -> store filter { userId, agentId, layer, scope?, key? }
|   agent-type -> store filter { userId, agentType, layer, scope?, key? }
|   user-global -> store filter { userId, layer, scope?, key? }
|-- status == "active" only, expiresAt > now -> PublicMemoryRecord[] + audit

write(req, policy)
|-- validateMemoryWriteRequest -> invalid -> deny "invalid_record: ..."
|-- layer == RUN_STATE      -> deny
|-- resolveWriteDecision(policy, layer, origin):
|   "deny"      -> rejected
|   "allow"     -> supersede prior active for (agent, layer, scope, key) -> expired
|                   insert status "active"
|   "approval"  -> insert status "pending_approval" (invisible to read)
|-- audit { decision, layer, scope, key, agentId, userId, reason? } + logger

approvePending(recordId, approver) // approver is a real user/admin id, NEVER an agent
|-- pending_approval -> supersede prior active -> mark "active"
```

Every op returns a structured `MemoryAudit` and emits a correlated `logger.child("agent-memory")` line.
`pending_approval` records are **never** returned by `read()`.

1.4 Persistence - AgentMemoryRecord (migration GENERATED, NOT APPLIED)

`prisma/schema.prisma`: +2 enums (`AgentMemoryLayer` - the 6 table layers; `AgentMemoryRecordStatus`), +1 model `AgentMemoryRecord` (`agentId`/`userId` bare indexed strings, `runId?`, `layer`, `scope`, `key`, `value` `Json`, `retention` `Json`, `provenance` `Json`, `status`, `expiresAt?`, `timestamps`, `deletedAt?`). Indexes: [`userId`], [`agentId`], [`userId`, `agentId`, `layer`, `scope`, `key`], [`userId`, `layer`, `scope`], [`status`], [`expiresAt`], [`deletedAt`].

Migration `20260906130000_add_agent_memory_record/migration.sql` - **GENERATED via prisma migrate diff (offline), hand-reviewed, header marks it NOT APPLIED**. Purely additive: 2 enums + 1 table, no DROP, no ALTER TABLE, never references `AgentMemory` / `Agent` / `User`. The legacy `AgentMemory` model (flat key/value, frozen legacy layer) is untouched.

1.5 Not wired into the runtime yet

A7 ships the **authority**, not the wiring. `tick()` does not read or write memory - no agent needs it until A11-A13.
`createMemoryGateway()` returns a `PrismaMemoryStore`-backed gateway that is inert until the migration is applied. This keeps `tick()` free of a new DB dependency at G07.

2. G07 proof - `npm run validate:agent-memory` -> 19 passed, 0 failed (offline)

2.1 Contract

<code>resolveWriteDecision</code> : default-deny for an ungranted layer	■
granted + <code>writePolicy</code> : <code>allow</code> -> <code>allow</code> ; granted + <code>deny</code> -> <code>deny</code>	■
knowledge layer + agent-derived -> forced approval (even with <code>writePolicy</code> : <code>allow</code>); <code>user/system</code> origin stays <code>allow</code>	■
<code>validateMemoryWriteRequest</code> rejects <code>RUN_STATE</code> , empty <code>key</code> , <code>undefined</code> value, > 16 KiB value, bad provenance origin, empty producer	■

2.2 Gateway - ALLOWED

```
policy grants SHORT_TERM (allow) -> write { value } -> status "active"
-> read (own scope) -> returns the record, audit.decision = "allow"
```

2.3 Gateway - REJECTED

case	result
layer not in policy.layers	read + write denied (not granted)
RUN_STATE	read + write denied (execution state, not memory)
value > 16 KiB	write rejected (invalid_record)
empty key	write rejected
writePolicy: deny	write denied

2.4 Gateway - SCOPE enforcement

readPolicy	agent B (same user)	outcome
own	different agentId	cannot see agent A's record; agent A can
agent-type	same type	sees it; different type does not
user-global	any agent	sees it; a different user sees nothing

2.5 Gateway - NEVER SILENTLY PROMOTE (the key rule)

```
policy: LONG_TERM granted, writePolicy "allow"
agent-derived write to LONG_TERM
-> decision "approval", status "pending_approval", audit.reason "approval required"
-> read -> 0 records (pending memory is never surfaced)
approvePending(id, "user:u1")
-> status "active" -> read -> 1 record
system-origin write to the SAME layer -> status "active" immediately (trusted, not agent-derived)
```

Plus: **supersede** (a second allow write of a key expires the prior active record - read returns only the latest); **TTL** (a record past its expiresAt is not returned).

2.6 Schema / migration parity

- generated AgentMemoryLayer enum **equals** TABLE_MEMORY_LAYERS (the 6; RUN_STATE excluded)
- migration present; header NOT APPLIED; forbids migrate dev; no DROP, no ALTER TABLE; creates exactly AgentMemoryRecord + the 2 enums; never names AgentMemory / Agent / User

2.7 Regression - no gate lost

validate:agent-contracts 38/0 - validate:agent-tools 20/0 - validate:agent-run-persistence 17/0 -
validate:agent-runtime 9/0 - validate:agent-supervisor 11/0 - validate:agent-integrity 21/0 -
validate:agent-memory **19/0**.

3. TypeScript

npx tsc --noEmit: **0 errors** in services/agent-framework/**, types/agent-framework/**,
scripts/validate-agent-*.ts, lib/generated/prisma/** (new AgentMemoryRecord model). Repo-wide 77, all in the
stale generated .next/dev/types/validator.ts (gitignored, pre-existing).

4. Files changed

Added (7):

```
frontend/services/agent-framework/memory/memory-store.ts
frontend/services/agent-framework/memory/in-memory-store.ts
frontend/services/agent-framework/memory/prisma-memory-store.ts
frontend/services/agent-framework/memory/memory-gateway.ts
frontend/services/agent-framework/memory/index.ts
frontend/prisma/migrations/20260906130000_add_agent_memory_record/migration.sql (GENERATED + REVIEWED, NOT APPLIED)
frontend/scripts/validate-agent-memory.ts
```

Modified (3):

```
frontend/types/agent-framework/memory-contract.ts + provenance/origin/status, TABLE_/KNOWLEDGE_ layers, MEMORY_POLICY_
frontend/prisma/schema.prisma + 2 enums, 1 model (appended; no existing line changed)
frontend/package.json + "validate:agent-memory"
```

Not touched: every existing model, the legacy AgentMemory / services/agents/* / /dashboard/agents, the AT24 Knowledge/RAG system, the runtime `tick()`, contracts A1-A6 behaviour. No API routes.

5. Non-goals honoured

No automatic knowledge promotion (the opposite - it's blocked) - no unrestricted RAG - no vector-search - no new Knowledge system - no Research Agent - no autonomous trading - no UI - no A9 credit ledger. A7 is the memory **authority**; runtime wiring and per-agent memory use come with A11-A13.

6. One point carried forward (owner's A6 note)

The owner asked that A7-A10 not each re-invent an interpretation of autonomy. A7 does **not** re-implement the `autonomyLevel < 2` trading-field rule - that stays in the integrity gate (A6). Memory governance here keys off the agent's `MemoryPolicy` (a distinct concern) and the record's `provenance.origin`. When A8 lands the full permission model, the memory gateway should consume the relevant permission result rather than adding its own new interpretation - the `resolveWriteDecision` seam is where that plugs in.

7. Status

A7 complete. The `MemoryGateway` is the single policy-gated authority for agent memory: default-deny, scope-enforced, size-bounded, provenance-aware, auditable, deterministic - and structurally unable to let an agent promote its own output into trusted knowledge without human approval. Migration is generated + reviewed, **not applied**.

Next per AN1.1 sec 7: A8 (Permissions / Guardrails deepening), A9 (Credit ledger), A10 (Evaluation + Observability), then A11-A13 (the three real agents).

G07 review requested - schema + `migration.sql` + the gateway.

End AN1.9.